

Summary

This video explains minimal Deterministic Finite Automata (DFA) construction techniques for infinite languages. The focus is on systematic “AD Rules” that allow rapid determination of the number of states required for common language patterns.

Key structural principle:

- Finite languages require acyclic productive structures.
- Infinite languages necessarily require loops or cycles among productive states.

Core AD Rules:

- Substring and positional constraints generally require $n + 2$ states.
 - Exactly n and At Most n counting constraints require $n + 2$ states.
 - At Least n constraints require $n + 1$ states.
 - Modulo constraints require exactly n states arranged in a cycle.
1. Fundamental Principles of DFA Construction

Finite vs Infinite Languages

Finite Languages

A DFA accepts a finite language if and only if its productive portion (states that can reach final states) is free from cycles.

Example:

Language of strings of length exactly 4 with equal number of a's and b's.

The DFA must:

- Track total length.
- Track symbol counts.
- Redirect all excess transitions to a dead state.

Such constructions require multiple distinct states and no productive loops.

Infinite Languages

To accept infinitely many strings, the DFA must contain at least one productive loop or cycle. Without such a structure, only finitely many strings can be generated.

Basic Universal Languages

Language	Description	Minimal States	Structure
Σ^*	All strings including ϵ	1	Single state, initial = final, self-loop on all symbols
Σ^+	All non-empty strings	2	Initial non-final $\rightarrow$ final; final has self-loop

2.

Substring and Positional Logic ($n + 2$ Rule)

When a language is defined by prefix constraints or fixed positions, the state count depends on the length of the constraint.

Starts With Constraint

If every string must start with a fixed substring of length n :

State Formula:

$$n + 2$$

Structural Breakdown:

- n states to process the required prefix.
- 1 final state with self-loop for arbitrary continuation.
- 1 dead state for deviation from prefix.

Examples:

Constraint	n	States Required
Starts with '0'	1	3
Starts with '101'	3	5

Fixed n -th Symbol from Left

If the n -th symbol from the left must be fixed:

State Formula:

$$n + 2$$

Structural Logic:

- States to track positions up to n .
- Transition to final state if symbol matches.
- Transition to dead state if symbol mismatches.

Examples:

Constraint	n	States Required
2nd symbol is '0'	2	4
3rd symbol is '1'	3	5

3.

Quantity-Based Constraints

When counting occurrences of a symbol x :

Constraint Type	Description	Minimal States
Exactly n	Accept only n , reject $n+1$	$n + 2$
At Most n	Accept 0 to n , reject $n+1$	$n + 2$
At Least n	Accept n and above	$n + 1$

Explanation

Exactly n :

- Need states for 0 to n .
- Need dead state for $n+1$.

At Most n :

- Same as Exactly n structurally.
- Dead state required for exceeding count.

At Least n :

- States 0 to n .
- The n -th state becomes final with self-loop.
- No separate dead state required for further occurrences of counted symbol.

4. Modulo Arithmetic and Cycle Logic

Modulo-based languages rely on cyclic transitions.

General Rule

If the number of occurrences of symbol x must satisfy:

$$x \equiv r \pmod{n}$$

Then:

Total states = n

Structural Design:

- States $Q_0, Q_1, \dots, Q_{n-1}$.
- On reading x :
 $Q_0 \rightarrow Q_1 \rightarrow Q_2 \rightarrow \dots \rightarrow Q_{n-1} \rightarrow Q_0$.
- Other symbols: self-loops on all states.
- Final state = Q_r .

Examples

Constraint	States	Final State
Even ($0 \bmod 2$)	2	Q_0
Odd ($1 \bmod 2$)	2	Q_1
$2 \bmod 3$	3	Q_2
$3 \bmod 4$	4	Q_3

This cyclic structure ensures infinite acceptance while maintaining minimality.

5. Summary Table of AD Rules

Language Category	Constraint	State Formula
Prefix	Starts with substring of length n	$n + 2$
Positional	n -th symbol from left fixed	$n + 2$
Exact Count	Number of $x = n$	$n + 2$
Maximum Count	Number of $x \leq n$	$n + 2$

Minimum Count	Number of $x \geq n$	$n + 1$
Modulo Constraint	Number of $x \equiv r \pmod{n}$	n

